Programación Concurrente

Trabajo Práctico

Las cámaras de eventos (también llamadas neuromórficas) son un tipo de sensor bio-inspirado que detecta cambios en la intensidad lumínica I en regiones puntuales del espacio (píxeles). La actividad en un píxel es registrada de forma asincrónica cuando $\log(I)$ supera cierto umbral. De esta manera, puede pensarse que este tipo de sensores detectan bordes en movimiento. Cuando un píxel detecta un cambio, se genera un evento que se puede representar como una tupla $\{(t, x, y, p)\}$ donde (x, y) es la posición del píxel y t es el *timestamp*, expresado en microsegundos, durante el cual ocurrió el cambio. El valor p indica la polaridad del cambio (si la intensidad aumentó o disminuyó por encima de un umbral fijo C).

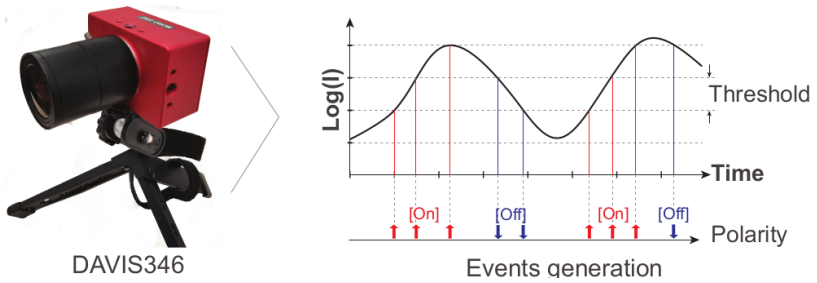


Figura 1: Una cámara de eventos y el modelo de generación de eventos para un píxel.

Finalmente, la salida de una cámara de eventos es una secuencia de eventos ordenada por *timestamp* que se estructura en algún archivo. Existen múltiples formas de visualizar estas secuencias, la más simple consiste en tomar ventanas temporales fijas y acumular los eventos en *event frames*.

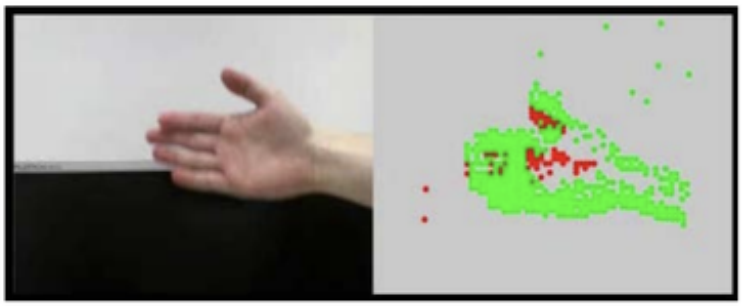


Figura 2: Visualización de eventos con event frame de 33 ms y distinguido por polaridad.

Una de las características distintivas de este tipo de cámaras es que tienen una resolución temporal muy alta (del orden de microsegundos), permitiendo capturar movimientos

muy rápidos sufriendo de poco *motion blur* a comparación de cámaras RGB tradicionales[1]. En este trabajo consideraremos un conjunto experimental de datos denominado PokerDVS capturados con una cámara de eventos de baja resolución (128x128 pixeles). Como puede observarse en esta muestra, las grabaciones se realizaron mostrando rápidamente frente a la cámara un mazo de naipes franceses usando una representación de *event frames*. Adicionalmente, en el trabajo original de PokerDVS se aplicó un método de detección para segmentar, espacial y temporalmente, los eventos correspondientes a los distintos símbolos (corazones, diamantes, tréboles y picas) que iban apareciendo en la grabación y, posteriormente, se extrajo cada símbolo detectado en un archivo separado. En este trabajo nos interesa analizar ciertas estadísticas simples del conjunto de datos agrupado por símbolos. Esto es útil porque permite un primer análisis exploratorio del dataset, permitiendo entender, entre otras cosas, posibles sesgos o características distintivas de cada símbolo. Quedará fuera del alcance de este trabajo la implementación de algoritmos que permitan, dado una secuencia de eventos, clasificarla en uno de los símbolos del conjunto de datos. En particular, queremos computar dos medidas que resumen el comportamiento temporal de cada símbolo: la media y la desviación estándar del inter-event interval (IEI). El IEI entre dos eventos se define como el tiempo transcurrido entre ellos. Registraremos una muestra, asociada a la población de un símbolo, cada vez que ocurra un evento, calculando el IEI con respecto al evento anterior. Luego de procesar todo el conjunto de datos agregaremos la información de la población de muestras de cada símbolo con la media poblacional:

$$\mu = \frac{1}{N_\mathrm{s}} \sum_{i=1}^{N_\mathrm{s}} x_i \tag{1}$$

y con el desvío estándar poblacional:

$$\sigma = \sqrt{\frac{1}{N_\mathrm{s}} \sum_{i=1}^{N_\mathrm{s}} (x_i - \mu)^2} \tag{2}$$

Donde N_s es la cantidad de muestras para un símbolo s, x_i es el valor de la muestra i asociada al símbolo s. De esta forma, se debería poder replicar la siguiente figura utilizando todos los archivos de símbolos del conjunto de datos PokerDVS.

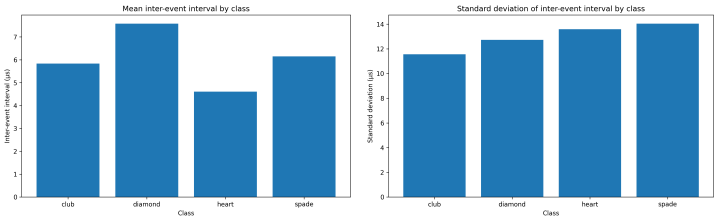


Figura 3: IEI medio y desviación estándar para cada símbolo del conjunto de datos PokerDVS.

Desde el punto de vista de la concurrencia, nuestro objetivo es conseguir una subdivisión del trabajo en tareas independientes que permitan una mejora en el tiempo de ejecución total del programa.

[1] Video ilustrativo

Objetivo del Trabajo Práctico

Se pide implementar un programa en Java que compute la media y la desviación estándar del IEI por símbolo, computado sobre todos los archivos del conjunto de datos [2]. El mismo consiste en archivos `.csv` separados por símbolos (*heart*=corazón, *club*=trébol, *diamond*=diamante, *spade*=pica), donde cada uno contiene una línea por evento con el siguiente formato: `timestamp,x,y,polarity`. En total, el dataset cuenta con 30 archivos de trébol, 43 de diamante, 23 de corazón y 35 de pica. Algunos archivos indican con una 'i' al final de su nombre que los símbolos estaban invertidos, no utilizaremos ese dato.

En cada archivo el IEI se calcula entre líneas consecutivas, calculando la diferencia de tiempo entre una medición y la siguiente (tomar en cuenta que a veces se detectan múltiples símbolos en el mismo momento, por lo que el IEI da 0, esta medición debe estar contabilizada). Los IEI se computan **por archivo** (no se combinan mediciones de archivos distintos, que pueden pertenecer a sesiones distintas). De un archivo con n líneas (descontando el encabezado) se obtendrán $n-1$ mediciones del IEI (la diferencia de tiempo de la segunda línea con la primera, la tercera con la segunda, ..., la última con la anteúltima). Puede asumir que el archivo `.csv` tiene el encabezado y al menos dos mediciones. La media y desvío estándar calculados, por supuesto, deben ser globales a todas las mediciones en todos los archivos.

El programa debe escribir los resultados en un archivo de salida `resultados.csv` que contenga una línea por cada clase, cada una con los siguientes campos (el encabezado debe estar incluído como primera línea del archivo de salida):

```
clase,archivos,intervalos,media_us,std_us
```

Donde:

`clase` es el símbolo (usar, como en las fuentes, los nombres en inglés *heart*, *club*, *diamond* y *spade*)

`archivos` es la cantidad de archivos procesados (en total para ese símbolo)

`intervalos` es la cantidad de mediciones tomadas en total entre todos los archivos (para ese símbolo).

`media_us` es la media calculada para los IEI

`std_us` es el desvío estándar calculado para los IEI

Tanto la operación de carga de datos como la de cómputo de estadísticas debe ser concurrente, dividiendo el trabajo entre una cantidad configurable de threads. Para el cálculo de la media, tener en cuenta que la cantidad de archivos por símbolo a procesar por cada thread podría no ser la misma. A su vez, observar que si bien, a priori, el cálculo concurrente del desvío estándar requiere restar la media a cada muestra, es posible utilizar la siguiente fórmula que se puede deducir de la ecuación 2:

$$\sigma = \sqrt{\frac{1}{N_\mathrm{s}} \sum_{i=1}^{N_\mathrm{s}} x_i^2 - \mu^2} \tag{3}$$

Para hacer pruebas, se puede armar un directorio con sólo parte de los archivos del set (debería dar resultados distintos), o incluso generar archivos `.csv` propios para testear casos particulares.

[2] El conjunto de datos puede ser descargado en formato `.zip` desde `https://drive.google.com/file/d/1CrpdQQq7P9guFf0i3z7AjexeJUKNLuvE/view?usp=sharing`

Elementos a entregar

Programa Java

Se debe presentar un programa escrito en Java con la siguiente estructura:

1. Una clase `Main` con el punto de entrada del programa, que tiene la responsabilidad de inicializar las estructuras mencionadas a continuación y producir unidades de trabajo que permitan procesar las estadísticas del dataset por lotes. El programa debe terminar únicamente cuando se haya generado el archivo de salida, y debe informar el tiempo transcurrido desde el inicio del procesamiento (justo antes de iniciar los threads) hasta el final del mismo (una vez que se calcularon los valores finales, sin incluir la escritura del archivo de salida).

2. Una clase `Buffer` (implementada como un monitor utilizando métodos synchronized) que actúa como una cola FIFO concurrente de capacidad acotada. Es decir, bloquea a un lector intentando sacar un elemento cuando está vacía y bloquea a un productor intentando agregar un elemento cuando está llena. La capacidad del Buffer también debe ser un parámetro configurable desde la clase `Main`.

3. Una clase `WorkerCounter` (implementada como un monitor utilizando métodos synchronized) que evita que `Main` termine su ejecución mientras queden threads trabajando.

4. Una clase abstracta `Task` que implementa `Runnable`, de la cual heredarán las tareas que sean ingresadas al buffer.

5. Una clase `StatsTask` que hereda de `Task`, y cuyo método `run` realiza el cálculo parcial de la estadística del lote asignado.

6. Una clase `StatsWorker` que hereda de `Thread` y que, en tanto esté activo, tome tareas disponibles en el buffer (obtenido en su creación) y las resuelva.

7. Una clase `PoisonPill` que hereda de `Task` y que hace que el `StatsWorker` que la tome termine su ejecución (puede ser por medio de una excepción, como en la práctica).

8. Una clase `ThreadPool`, que se encarga de instanciar e iniciar los `StatsWorker` solicitados.

9. Cualquier otra clase que considere necesaria.

Al iniciar el programa la clase `Main` debe leer el directorio de archivos `.csv` y realizar la división en lotes. Luego, debe delegar la iniciación de los threads necesarios en la clase `ThreadPool` e introducir de a una las `StatsTask` que indiquen el lote de archivos a procesar, en el `Buffer` (y, cuando sea adecuado, también introducir las `PoisonPills` para indicar la finalización de los `StatsWorker`s) El tamaño del lote, expresado en cantidad de archivos, es un valor a experimentar y que tendrá un impacto en el tiempo de ejecución del programa. Notar que no todos los archivos tienen la misma cantidad de eventos, por lo que una distribución equitativa en cantidad de archivos por Thread no garantiza una carga necesariamente balanceada. Cada `StatsWorker` en funcionamiento debe tomar del `Buffer` una `Task` y computar el cálculo de las estadísticas necesarias para ese lote (separándolas por tipo de símbolo). Se debe crear la cantidad de `Worker`s solicitada por el usuario, y todos deben cooperar (salvo que la cantidad de lotes sea menor que la cantidad de `Worker`s, aunque no sería un escenario común en la práctica). Cada `StatsWorker` debe publicar los resultados parciales utilizando algún tipo de estructura de datos compartida,

y al finalizar el procesamiento de todos los lotes, la clase `Main` debe recolectar todos los resultados parciales y producir el resultado final en el archivo `.csv` de salida. Para el cómputo de los valores, utilice valores de tipo `double` (o `long` para enteros). Los números finales deben tener 6 dígitos decimales de precisión. Para la estructura de datos, no está permitido utilizar ninguna clase de biblioteca de Java que resuelva la concurrencia directamente (de requerir una funcionalidad distinta a las descritas en este enunciado, impleméntela usando los mencanismos de de Java de monitores vistos en clase). El programa, además de funcionar adecuadamente, debe estar **correctamente documentado**. El programa debe poder ejecutarse por línea de comandos recibiendo por parámetro: el directorio de entrada, el tamaño del buffer, la cantidad de `Workers`, y el tamaño de los lotes en ese orden. No es incorrecto que el programa haga impresiones por pantalla intermedias para informar su progreso, pero se pide evitar la impresión de mensajes de *debug*, al menos en una ejecución por defecto.

Informe

Además del código, se requiere un informe en formato `pdf`, donde se debe mostrar el resultado del programa, replicando la figura 3 (es decir, gráficos de barras con una barra por símbolo cuyas alturas son el IEI medio en un gráfico, y el desvío estándar en el otro). Se debe registrar el tiempo de ejecución del programa utilizando distintas combinaciones de cantidades de threads y tamaños de lotes:

- **Tamaño Lote:** 1, 16, 32, 64 y 130
- **Cantidad de Threads:** 1, 4, 8 y 16

  El tiempo de ejecución debe medir la sección comprendida justo antes del inicio de los workers y justo después de que el main haya obtenido el resultado final.

El tamaño del buffer debe dejarse fijo en todas las pruebas, pero sugerimos que se experimente con distintos valores que permitan conseguir un buen rendimiento.

El informe, que es **condición necesaria para la aprobación del TP**, debe respetar el siguiente formato:

1. **Autoría:** Nombre, e-mail y número de legajo (o DNI) de quienes hicieron el trabajo (3 personas salvo excepciones acordadas con los docentes).
2. **Introducción:** Sección explicando el dominio del TP y la motivación del trabajo, junto con el diseño del código y cualquier consideración adicional que considere relevante para la correcta interpretación de los resultados. En esta sección es importante que se **describa el funcionamiento del programa**, explicando qué clases lo constituyen, como interactúan y **las decisiones de diseño tomadas** al crearlo (Una vez que inicia, ¿Cómo reparte los archivos? ¿Dónde se guardan los resultados parciales? ¿Cómo se coordinan los threads para actualizarlos?). También debe tener las **instrucciones de uso** del programa, explicando a alguien que quiera usarlo cuáles y en qué orden van los parámetros y cómo es su formato.
3. **Evaluación:** Sección explicando el proceso de evaluación, incluyendo los detalles del hardware donde se ejecutan las pruebas (modelo de microprocesador, memoria disponible y versión del sistema operativo; para que la comparación tenga sentido todas las pruebas deben hacerse en el mismo equipo). En esta sección debe incluirse una hipótesis, que resuma el comportamiento esperado por ustedes al variar la cantidad de threads y el tamaño de los lotes. Esto debe estar fundado en la experiencia adquirida durante la implementación del programa, la teoría vista en clase e intuiciones discutidas entre el grupo. Luego se deben presentar tablas

reportando los tiempos de ejecución para cada combinación de Tamaño de lote y Cantidad de Threads, más uno o más gráficos que muestren el tiempo de ejecución en función de la cantidad de threads para los distintos tamaños de lote. Es decir, el eje x representará la cantidad de threads y el eje y el tiempo de ejecución. Esto puede representarse en un gráfico por cada tamaño de lote, o bien en un solo gráfico con distintas curvas para cada tamaño de lote. Para tomar los tiempos de ejecución, se recomienda ejecutar cada combinación una gran cantidad de veces, desechar las primeras 10 corridas y luego promediar los tiempos de ejecución de las corridas restantes. Esto permite reducir el efecto de la variabilidad en los tiempos de ejecución, que puede ser causada por distintos factores (por ejemplo, la carga del sistema operativo, *warm-up* de la JVM, etc.). Ya sea que tomen este promedio o que opten por otra manera de tomar los resultados, **todo procesamiento de los datos tiene que estar documentado en el informe**.

4. **Análisis:** Sección en la que se debe discutir los datos obtenidos en la evaluación, en particular destacando la combinación con la que se obtuvo el tiempo de ejecución mínimo y la combinación que obtuvo el tiempo máximo para cada tamaño, más alguna conclusión (Por ejemplo, responder a ¿Es verdad que aumentar la cantidad de threads siempre mejora el rendimiento? ¿El comportamiento al aumentar la cantidad de threads es el mismo cuando cambia el tamaño del lote? ¿Cuál considera que es la razón por la que se observaron estos resultados?). Es importante que aquí **se contraste la hipótesis presentada anteriormente con los resultados obtenidos**.

Forma de Entrega

La entrega se realizará por medio de Discord. Para esto, en primer lugar debe registrarse el grupo llenando el formulario en `https://forms.gle/oGRe1mWpG6mnvi1v7`. Hecho esto, se creará un canal de Discord exclusivo para el grupo donde podrán hacer consultas y contactar al plantel docente. Al finalizar el TP, se debe subir el archivo `.zip` con el código del programa, los archivos de texto utilizados, y el `.pdf` con el informe al canal correspondiente al grupo.

Es posible trabajar en un repositorio *git* compartido por las personas que compongan el grupo. En este caso, el repositorio debe ser **privado**. Al entregar, se deberá agregar a los profesores al repositorio y publicar su link en el canal del grupo en el Discord de la materia. Se aclara que se espera que el código entregado por el grupo sea **original**. Si se detecta que el mismo fue copiado de alguna fuente online o que fue enteramente generado por un modelo de lenguaje, será automáticamente desaprobado sin posibilidad de reentrega.

Fecha de Entrega y Coloquio

El TP debe entregarse antes del **Domingo 8 de Noviembre** a las **23:59hs**. En caso de ser necesario, se solicitará una reentrega antes el **Domingo 29 de Noviembre** a las **23:59hs**. La reentrega consistirá en la corrección de los elementos que se hayan marcado como incorrectos de la entrega original. **Se pide encarecidamente que si pueden entregar antes de la fecha límite, lo hagan**.

Luego de la entrega, los días **Sábado 14 de Noviembre** y **Miércoles 18 de Noviembre** en el horario de clase se citará a las personas integrantes del grupo para un coloquio donde se pedirá que expliquen cómo fue el proceso de implementación del TP. Se acordará una franja de 25 minutos dentro del rango con cada grupo individualmente con este propósito.

Apéndice: Algunas herramientas de Java

Medición de tiempos

Para medir el tiempo de ejecución del programa se puede llamar al método provisto por Java `System.nanoTime()`. Se debe guardar el tiempo actual cuando se va a comenzar a operar y cuando se terminó la ejecución (es decir, luego de que los Workers hayan parado y el Main obtenido el resultado final). Luego, basta con hacer la diferencia entre ambos tiempos.